Intent Canonicalization & Hashing Protocol

Public Technical Disclosure – Prior Art Publication

Publication Date: January 8, 2026

Status: Defensive Publication for Prior Art

Abstract

This document discloses a deterministic method for canonicalizing autonomous agent action intents and computing cryptographically verifiable content-addressed identifiers ("intent hashes"). The intent hash enables cryptographic binding between what an agent proposes to do and what authorization tokens permit, supporting auditability, non-repudiation, and tamper detection.[1][2]

This specification is published as prior art to establish the canonical intent format for AI agent authorization systems and prevent patent enclosure of these foundational patterns.[2]

---

1. Problem Statement

When autonomous agents (AI systems, bots, automated tools) execute actions on behalf of users, a fundamental security requirement exists: **the action executed must match exactly what was authorized**.[2]

Without deterministic canonicalization:

- Different JSON representations of the same intent produce different hashes
- Attackers can modify parameters between authorization and execution
- Audit trails become unreliable
- Policy enforcement becomes ambiguous

---

2. ActionIntent Data Structure

An Intent is a structured declaration of a proposed action containing:[2]

```
{
  "intent_version": "1.0",
  "intent_id": "int_01HXYZ...",
  "timestamp": "2026-01-08T17:04:00.000Z",
  "subject": {
    "subject_id": "usr_abc123",
    "subject_type": "user",
    "session_id": "sess_xyz789"
  },
  "tool": "email",
  "action": "send",
  "params": {
    "to": ["recipient@example.com"],
    "subject": "Meeting confirmation",
    "body_hash": "sha256:a1b2c3d4..."
  },
  "evidence_refs": ["ev_mfa_proof_123", "ev_user_consent_456"],
  "commitments": {
    "max_recipients": "5",
    "audit_required": "true"
  },
  "constraints": {
```

```
    "expires_at": "2026-01-08T17:09:00.000Z",
    "single_use": true
  }
}
```

2.1 Required Fields

Field	Type	Description
`intent_version`	string	Schema version (currently "1.0")
`subject`	object	Entity requesting the action
`tool`	string	Tool namespace (email, web, payments, etc.)
`action`	string	Specific operation (send, fetch, transfer)
`params`	object	Tool-specific parameters

2.2 Tool-Specific Parameter Examples

Web navigation:

```
{"url": "https://example.com", "method": "GET", "max_bytes": 2000000}
```

Email sending:

```
{"to": ["alice@example.com"], "subject": "...", "body_hash": "sha256:..."}
```

Payment transfer:

```
{"amount": "100.00", "currency": "USD", "to_account": "acct_xyz"}
```

---

3. Canonicalization Algorithm

To produce a deterministic hash, the JSON must be canonicalized using **RFC 8785 JSON Canonicalization Scheme (JCS)**.[2]

3.1 Algorithm Steps

1. **Parse** the Intent JSON into a native data structure
2. **Validate** against I-JSON constraints (RFC 7493)
3. **Serialize** using JCS rules:
    - Sort all object keys lexicographically (Unicode code point order)
    - Serialize numbers using ECMAScript semantics
    - Serialize strings with minimal escaping
    - Remove all insignificant whitespace
    - Use UTF-8 encoding
4. **Output** the canonical UTF-8 byte sequence

3.2 Key Sorting Example

Input:

```
{"z": 1, "a": 2, "m": {"b": 3, "a": 4}}
```

Canonical Output:

```
{"a":2,"m":{"a":4,"b":3},"z":1}
```

3.3 Number Serialization

Input	Canonical Output
`1.0`	`1`
`0.5`	`0.5`
`1e10`	`10000000000`
`1.5e-3`	`0.0015`

4. Hash Computation

The intent hash is computed as:[2]

```
intent_hash = "sha256:" + SHA256(canonicalize(intent))
```

4.1 Properties

- **Deterministic:** Same intent always produces same hash
- **Collision-resistant:** Different intents produce different hashes
- **Content-addressable:** The hash serves as a unique identifier
- **Tamper-evident:** Any modification changes the hash

4.2 Worked Example

Input Intent:

```
{
  "intent_version": "1.0",
  "subject": {"subject_id": "usr123", "subject_type": "user"},
  "tool": "email",
  "action": "send",
  "params": {"to": "test@example.com"}
}
```

Canonical Form:

```
{"action":"send","intent_version":"1.0","params":{"to":"test@example.com"},"subject":{"s
ubject_id":"usr123","subject_type":"user"},"tool":"email"}
```

Hash:

```
sha256:7a8b9c0d1e2f3a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d9e0f1a2b3c4d5e6f7a8b
```

5. Reference Implementation

```
import json
import hashlib

def canonicalize(obj):
    """Canonicalize a JSON-serializable object per RFC 8785 (JCS)."""
    if obj is None:
        return "null"
    elif isinstance(obj, bool):
        return "true" if obj else "false"
    elif isinstance(obj, int):
        return str(obj)
    elif isinstance(obj, float):
        if obj == int(obj):
            return str(int(obj))
        return repr(obj)
    elif isinstance(obj, str):
        return json.dumps(obj)
    elif isinstance(obj, list):
        items = [canonicalize(item) for item in obj]
        return "[" + ",".join(items) + "]"
    elif isinstance(obj, dict):
        sorted_keys = sorted(obj.keys())
        items = [json.dumps(k) + ":" + canonicalize(obj[k]) for k in sorted_keys]
        return "{" + ",".join(items) + "}"

def hash_intent(intent: dict) -> str:
    """Produce the canonical hash of an Intent."""
    canonical = canonicalize(intent)
    canonical_bytes = canonical.encode("utf-8")
    digest = hashlib.sha256(canonical_bytes).hexdigest()
    return f"sha256:{digest}"
```

5.1 Production Libraries

Language	Library	Source
Python	`jcs`	pypi.org/project/jcs

JavaScript	`json-canonicalize`	npmjs.com/package/json-canonicalize
Rust	`jcs`	docs.rs/jcs
Go	`go-jcs`	github.com/cyberphone/json-canonicalization

---

6. Use in Authorization Binding

The intent hash is bound to capability tokens, creating an unbreakable link between what was requested and what was authorized:[1][2]

```
{
  "token_id": "tok_abc123",
  "intent_hash": "sha256:7a8b9c0d...",
  "intent_snapshot": { /* full intent object */ },
  "decision": "ALLOW",
  "expires_at": "2026-01-08T17:09:00.000Z"
}
```

Verification flow:

1. Receive token with `intent_hash`
2. Reconstruct hash from `intent_snapshot`
3. Compare reconstructed hash with claimed `intent_hash`
4. If mismatch → Token invalid (tampering detected)
5. If match → Proceed with signature verification

---

7. Security Considerations

7.1 Hash Algorithm

SHA-256 is REQUIRED. Future versions may support SHA-3 or BLAKE3 via algorithm prefixes.[2]

7.2 Parameter Redaction

Sensitive parameters SHOULD be hashed rather than included verbatim:

```
{"authorization_header_hash": "sha256:abc123...", "body_hash": "sha256:def456..."}
```

7.3 Replay Protection

Intents SHOULD include:

- `idempotency_key` for deduplication
- `expires_at` for time-bounding
- `single_use: true` for one-time authorization

---

8. Related Work & Standards

- **RFC 8785:** JSON Canonicalization Scheme
- **RFC 7493:** I-JSON message format
- **UCAN Specification:** Capability tokens (delegation-focused)
- **Schema.org Actions:** Semantic action vocabulary
- **Content-Addressable Storage:** Hash-based identification patterns

---

9. Prior Art Statement

This specification is published as prior art under **CC0 1.0 Universal Public Domain Dedication** to:

1. Establish the canonical intent format for agent authorization systems
2. Prevent patent enclosure of structured intent formats for AI tools
3. Enable third parties to implement compatible systems
4. Support interoperability across agent authorization platforms

What this document covers (PUBLIC):

- Intent data structure and field definitions
- Canonicalization algorithm (RFC 8785)
- Hash computation method
- Token binding structure
- Verification flow

What remains proprietary (NOT DISCLOSED):

- How intents are evaluated (risk classification)
- How policy decisions are made
- How tokens are issued (signing algorithms, key management)
- How evidence is validated (scoring heuristics)

---

10. Revision History

Version	Date	Changes
1.0	January 8, 2026	Initial specification

End of Specification

How does this look? Would you like me to:

1. Add more test vectors for implementers?
2. Expand the tool-specific parameter schemas?
3. Add a formal ABNF grammar for the schema?
4. Prepare this in arXiv-ready LaTeX format?

⁂

---

1. lets-discuss-licensing-open-so-HV_bbJ5uQlOPuXCF7b61YQ.md

2. LProof-Intent-Schema-Specification.pdf
3. Adapter Coordination Interface Specification.pdf
4. Audit Logs_ Event Schema & Sanitized Examples.pdf
5. Conscious Sovereignty Layer (CSL) Solver.pdf
6. ΛProof Protocols.pdf
7. DAO Charter.pdf
8. Building a ΛDapp_ A Developer's Implementation Guide.pdf
9. Capability Token Semantic Mapping.pdf
10. Data Sovereignty Informational Personhood.pdf
11. ΛProof_ An Architectural Blueprint for Verifiable Artificial Intelligence.pdf
12. Λᴾ-Archivum.pdf
13. ΛProof - R96 Credits.pdf
14. Governance for a Non-Surveillance Digital Ecosystem.pdf
15. Token Specification.pdf
16. Risk Classification.pdf
17. Policy Rules Specification.pdf
18. Evidence Validation.pdf
19. How Trust is Measured.pdf
20. ΛProof Token Issuance.pdf